RICE UNIVERSITY

Sharding the State: ZeRO and FSDP

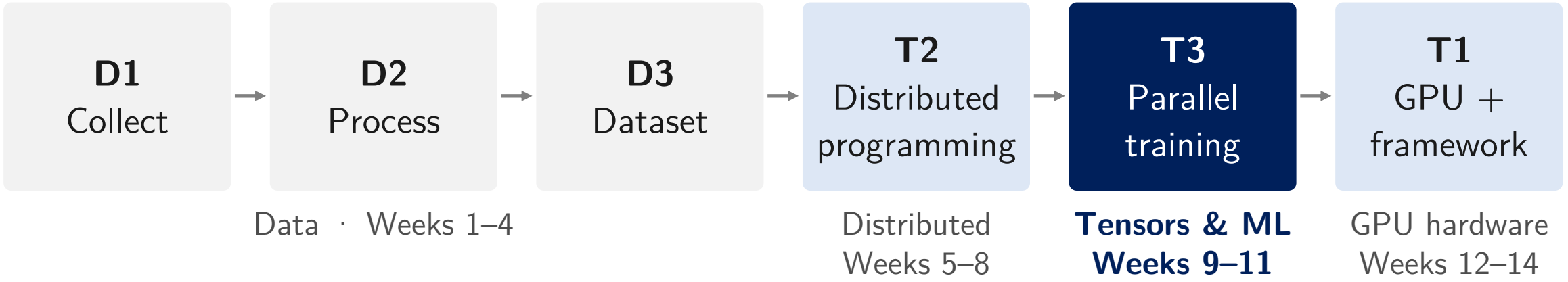
Stop storing N copies: shard the optimizer state, the gradients, and the parameters

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 9, Lecture 2

Where we are in the pipeline



In our pipeline

Today: stop storing **N copies** of the training state. This week: **T2 checkpoint** and **Lab 4 due**; then spring break.

Lecture 1: copy the model, split the batch → today: shard the copies → Week 10: split the model

Why do we need to shard the state?

One server, 8 GPUs, plain data parallelism
model states: **48 GB on every GPU** ($\Psi = 3 \times 10^9$, Lecture 1)

Of the 384 GB of model states in the server, how much is distinct?

GPU 0	GPU 1	GPU 2	GPU 3	GPU 4	GPU 5	GPU 6	GPU 7

distinct: 48 GB **copies: 336 GB**

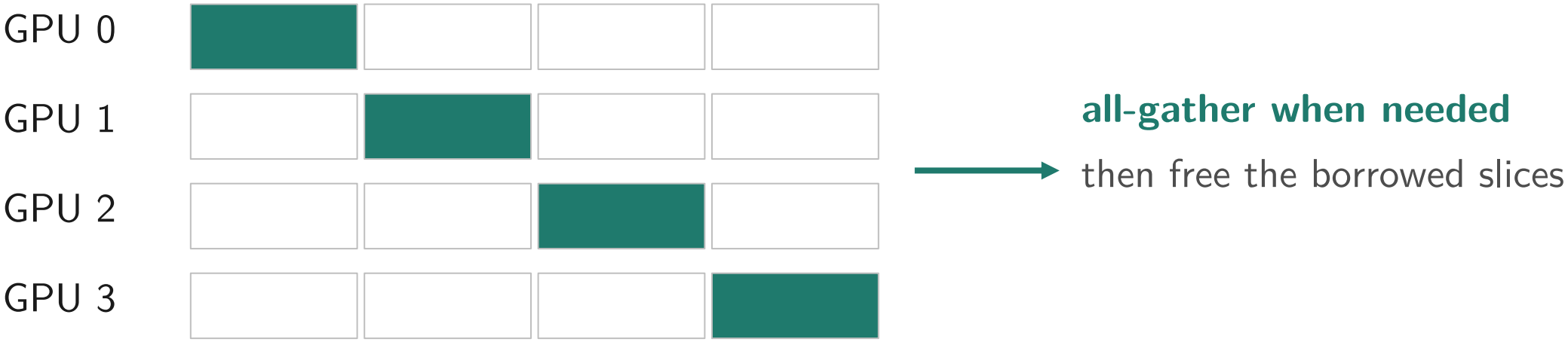
Answer

Only **1/N** is distinct: here **87.5%** of the model-state memory holds copies.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022). (80 GB)

Sharding, in plain words

Shard a state: cut it into N slices; GPU k **keeps only slice k** and fetches the others when it needs them.



Key idea

Still data parallelism — each GPU trains on its own examples — but the N **copies** become N **slices**.

Where sharding shows up

- **ZeRO** (SC 2020): the three stages we study today.
- **PyTorch FSDP**: PyTorch's built-in version of ZeRO stage 3.
- **Large runs**: e.g. Llama 3 used FSDP as its data-parallel split.
- **Costs**: up to **1.5×** the communication, more to debug.

In our pipeline

Lab 5 and T3: per-rank memory estimates use today's arithmetic.

Why do we need sharding?

Why shard? So the N GPUs hold **one** copy of the training state between them, not N — and bigger models fit.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020.

Learning goals for today

- **Explain** which state each GPU needs **when**, and why that lets ZeRO shard it.
- **Compute** per-GPU memory for plain DP and **ZeRO stages 1, 2, 3**.
- **Count** the traffic per step: the **same** as DP for stages 1–2, **1.5×** for stage 3.
- **Decide** when sharding pays, and how **FSDP** hides its traffic behind compute.

A puzzle to start

The model
7.5 billion parameters
(numbers that training learns)

The cluster
64 GPUs
80 GB of memory each

Train it with Adam; every GPU keeps a full copy. Does it fit?

Answer

No. Each GPU needs $\approx$ **120 GB** of training state — more GPUs do not help. Split 64 ways, it is $\approx$ **1.9 GB** per GPU.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020.

Recap: memory per GPU

Ψ = number of parameters, N = number of GPUs

one GPU



params 2Ψ grads 2Ψ optimizer state 12Ψ

$$\text{per-GPU memory} = 16\Psi + A \;/\; N$$

Lecture 1

A/N shrinks with more GPUs; **16Ψ does not**. Today: shrink the 16Ψ.

Recap: reduce-scatter and all-gather

- **reduce-scatter**: GPU k ends with the **sum of slice k** only.
- **all-gather**: every GPU ends with **all N slices**.
- **all-reduce** = reduce-scatter + all-gather.

$$\text{each GPU sends } \; (N-1)/N \cdot S \approx S \text{ bytes per phase}$$

Weeks 7–8

An all-reduce of S bytes costs about **2S** per GPU: **S** for each half.

Part 1

ZeRO stages 1 and 2

Which copies can a GPU give up — optimizer state, gradients — without sending more data?

Who needs what, and when?

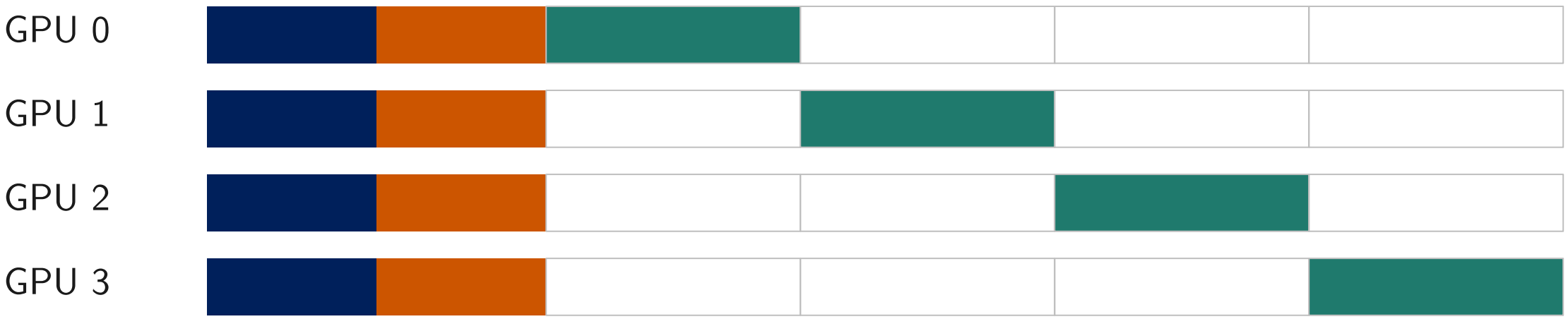
State	Bytes	Used in forward/backward?	Used in the update?
Parameters (16-bit)	2Ψ	**yes**, all of them	yes
Gradients (16-bit)	2Ψ	produced in backward	yes
Optimizer state	12Ψ	**no**	yes

Key idea

The update is **elementwise**: slice k of the parameters needs only slice k of the gradient and of the optimizer state.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Section 5.

ZeRO-1: shard the optimizer state



blue: params 2Ψ · orange: grads 2Ψ · green: optimizer state, slice k only

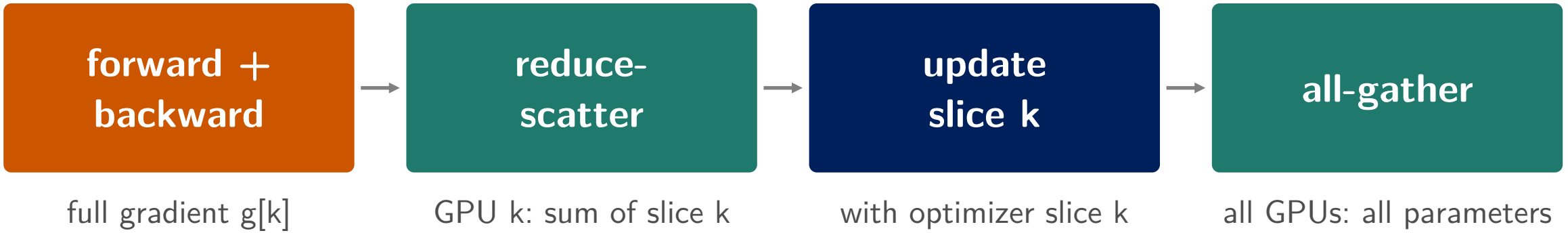
$$\text{ZeRO-1:} \quad 2\Psi + 2\Psi + 12\Psi/N = 4\Psi + 12\Psi/N$$

Check

N = 4: **7 bytes** per parameter instead of 16.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Section 5.1.

A ZeRO-1 step



Key idea

Reduce-scatter **before** the update, all-gather **after** it: the all-reduce of Lecture 1, cut in two.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Sections 5.1 and 7.2.

ZeRO-1 sends no extra data

$$\text{DP: all-reduce} = 2\,(N-1)/N \cdot \mathcal{S}$$

$$\text{ZeRO-1: } (N-1)/N \cdot \mathcal{S} + (N-1)/N \cdot \mathcal{S}$$

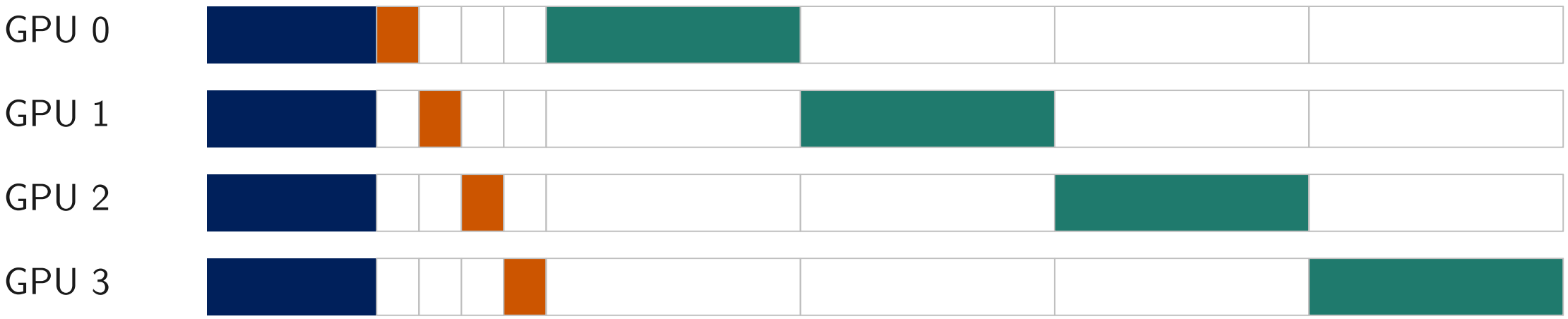
$$\mathcal{S} = 2\Psi \text{ bytes (bf16)} \quad \Rightarrow \quad \text{both} \approx 2\mathcal{S} = 4\Psi \text{ bytes per GPU}$$

Check

Same bytes as plain DP: ZeRO-1's memory saving costs **no extra communication**.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Section 7.2.1.

ZeRO-2: also drop the gradients you do not own



orange gradients and green optimizer state: slice k only

$$\text{ZeRO-2:} \quad 2\Psi + (2\Psi + 12\Psi)/N = \ 2\Psi + 14\Psi/N$$

Check

N = 4: **5.5 bytes** per parameter — and still the same traffic as DP.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Section 5.2.

The floor of stages 1 and 2

$$N \to \infty: \quad \text{ZeRO-1} \to 4\Psi, \qquad \text{ZeRO-2} \to 2\Psi$$

$\Psi = 7.5 \times 10^9$: ZeRO-1 never below **30 GB**
ZeRO-2 never below **15 GB**

Watch out

The **whole** parameters set a floor. To keep shrinking with N, shard the parameters too.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Section 5.

Check yourself

Two minutes with a neighbour.

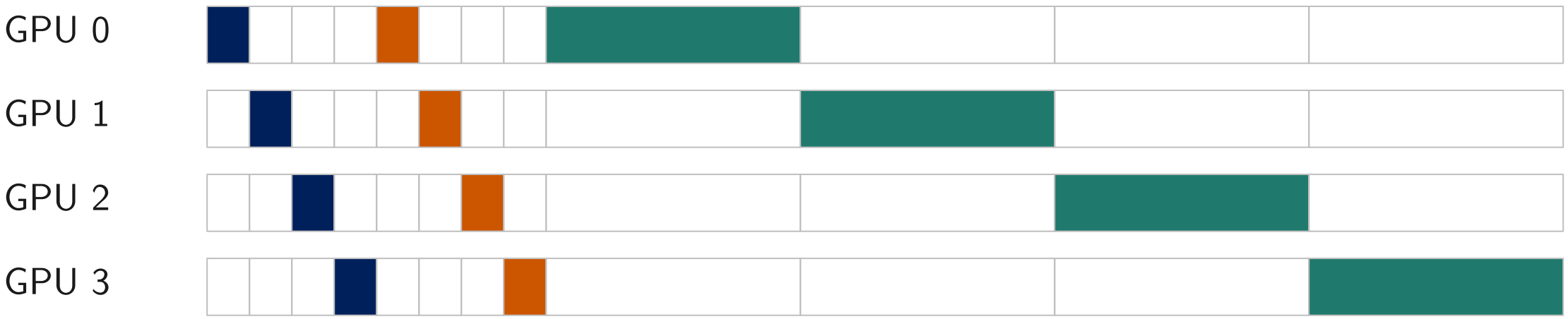
- Why may GPU k throw away 3/4 of the optimizer state when $N = 4$?
- ZeRO-2, $N = 8$, $\Psi = 1$ billion: how many GB of model state per GPU?
- Why does ZeRO-1 send no more data than plain data parallelism?

Part 2

ZeRO-3 and FSDP

Shard everything: what if each GPU keeps only 1/N of the parameters too?

ZeRO-3: shard the parameters too



every state: slice k only

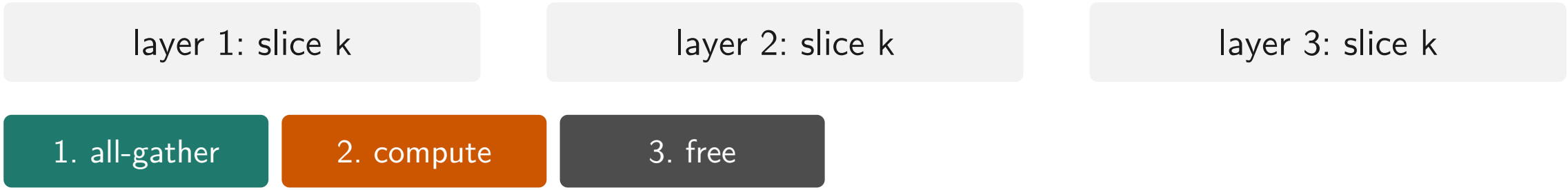
$$\text{ZeRO-3:} \quad (2\Psi + 2\Psi + 12\Psi)/N = 16\Psi/N$$

Watch out

Puzzle: 64 GPUs, $\approx$ **1.9 GB** each. But forward needs **whole** layers.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Section 5.3.

Parameters on demand



forward: for each layer, in order

backward: gather each layer **again** (in reverse), compute, **reduce-scatter** its gradients

Key idea

Only about **one layer** is whole at a time: the extra memory is one layer, not the model.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Section 5.3.

ZeRO-3 sends 1.5× the data

$$\text{forward: all-gather parameters} \approx \mathcal{S}$$

$$\text{backward: all-gather parameters again} \approx \mathcal{S}$$

$$\text{gradients: reduce-scatter} \approx \mathcal{S}$$

Memory

ZeRO-3: **3S** per step vs **2S** for DP, ZeRO-1 and ZeRO-2 — the price of dividing **everything** by N.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Section 7.2.2.

The three stages, to scale

one GPU of N = 4, to scale



Memory

Only **stage 3** keeps shrinking as N grows.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Figure 1.

FSDP: PyTorch's ZeRO-3

forward



free after use

backward



units 0, 1, 2 = embedding, hidden layer, output layer

AG: all-gather a unit's parameters · RS: reduce-scatter its gradients

Key idea

Gather a unit **just before** it is used, free it **right after**: only about one unit is whole at a time.

Source: PyTorch documentation, torch.distributed.fsdp.fully_shard (FSDP2); Zhao et al., "PyTorch FSDP," VLDB 2023.

FSDP in code

```
model = MLP().cuda()
for block in model.blocks:
    fully_shard(block)      # one unit per block
fully_shard(model)          # the root: the rest
opt = torch.optim.AdamW(model.parameters())
for x, y in loader:         # this rank's shard
    F.cross_entropy(model(x), y).backward()
    opt.step(); opt.zero_grad()
```

- Wrap **blocks**, then the root.
- Optimizer **after** sharding.
- Loop: unchanged.

Key idea

Like DDP, FSDP is a wrapper: the training loop stays the same; **where the bytes live** changes.

Source: PyTorch documentation, torch.distributed.fsdp.fully_shard (FSDP2); Zhao et al., "PyTorch FSDP," VLDB 2023.

How big should a unit be?

Unit size	Peak memory	Collectives per step
small (one layer)	low	**many**: latency adds up
large (whole model)	**high**: like DP	few

Key idea

Usual choice: **one unit per repeated block** of the model — big enough to amortize α, small enough to keep the peak low.

Source: PyTorch documentation, torch.distributed.fsdp.fully_shard (FSDP2); Zhao et al., "PyTorch FSDP," VLDB 2023.

Check yourself

Two minutes with a neighbour.

- ZeRO-3, $\Psi = 7.5 \times 10^9$, N $= 16$: model state per GPU?
- Why does ZeRO-3 gather each layer's parameters **twice** per step?
- What do you trade when you make FSDP's units smaller?

Part 3

Memory versus communication

When is sharding worth its traffic?

Memory and traffic, side by side

Method	Memory per GPU	Traffic per GPU
DP	16Ψ	2S
ZeRO-1 / ZeRO-2	$4\Psi + 12\Psi/\mathrm{N}$ / $2\Psi + 14\Psi/\mathrm{N}$	2S
ZeRO-3 (FSDP)	$16\Psi/\mathrm{N}$	**3S**

$\mathrm{S} = 2\Psi$ bytes: the bf16 parameters or gradients

Check

Stages 1–2 save memory **for free**. Stage 3 divides everything by N for **+50%** traffic.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Sections 5 and 7.2.

Back to the puzzle

$\Psi = 7.5 \times 10^9$ parameters, N = 64 GPUs of 80 GB

Method	Model state per GPU	Fits?
Plain data parallelism	120 GB	no
ZeRO-2	16.6 GB	yes
ZeRO-3	1.9 GB	yes

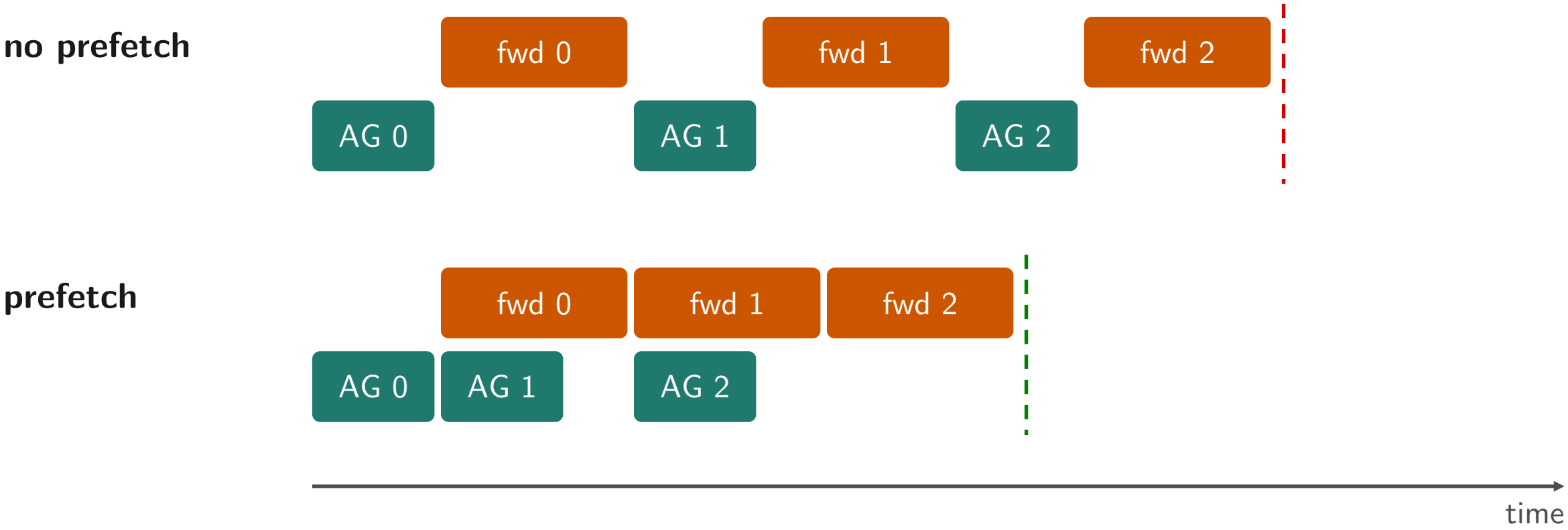
Answer

Sharding turns an impossible job into one that fits — the GPUs stop storing **copies**.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Figure 1.

Prefetch: gather the next unit early

AG: all-gather a unit's parameters · fwd: its forward

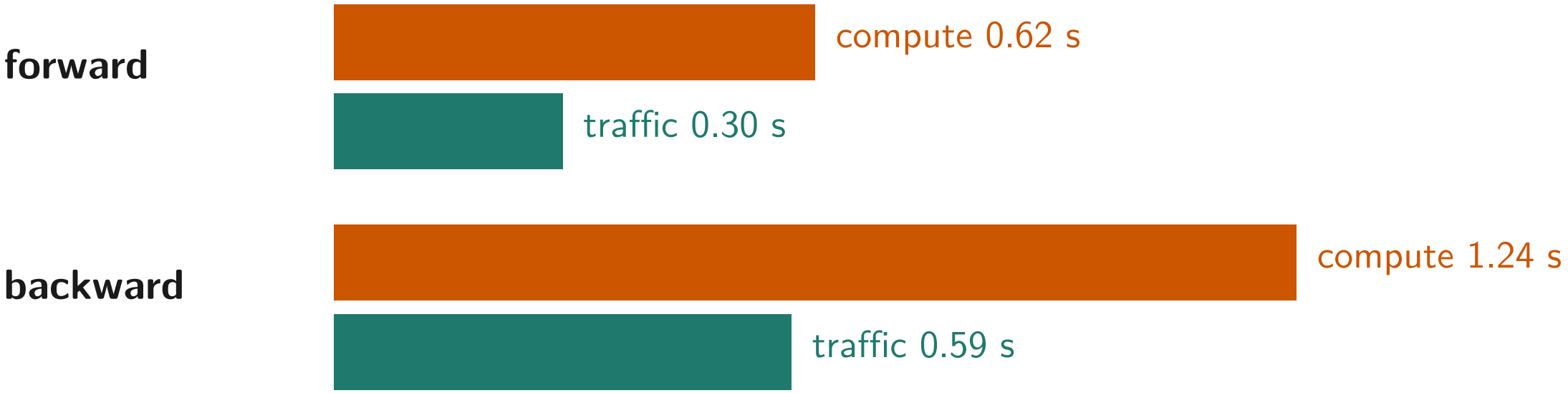


Key idea

While unit u computes, unit u + 1 **arrives**: only the first gather is exposed.

Source: PyTorch documentation, torch.distributed.fsdp.fully_shard (FSDP2); Zhao et al., "PyTorch FSDP," VLDB 2023.

Can the traffic hide?



Memory

Here the traffic is **shorter** than the compute it overlaps: hidden. Shrink the batch per GPU and it **stops** hiding.

Source: DGX H100 User Guide (400 Gb/s per GPU); H100 datasheet (bf16 peak, assumed 40%); 6 FLOP/parameter/token.

When sharding does not pay

- **The model already fits**: our MLP's states are 0.55 MB; sharding only adds collectives.
- **Little compute per step**: the extra traffic has nothing to hide behind.
- **A slow network**: 1.5× of a large transfer time.

Watch out

Shard only what does not fit: memory you do not need is not worth extra communication.

Choosing a stage

Does plain DP fit?	yes →	use **DDP**
Does ZeRO-2 fit?	yes →	use **ZeRO-1/2**: same traffic
Does ZeRO-3 fit?	yes →	use **ZeRO-3 / FSDP**: 1.5× traffic
Still no?	→	split the model: **Week 10**

Key idea

Pick the **cheapest** stage that fits, counting activations and headroom.

Check yourself

Two minutes with a neighbour.

- Why is FSDP a bad idea for our 34,338-parameter MLP?
- Your model fits with plain DP. Is there any reason to use ZeRO-2?
- The network gets 10$\times$ slower. Which stage suffers most, and why?

IN-CLASS BUILD

Memory and traffic for each choice

Work in pairs: one model, one server, four ways to train it.

Exercise A: memory per GPU

$$\text{DP: } 16\Psi = ___ \text{ GB}$$

$$\text{ZeRO-1: } 4\Psi + 12\Psi/N = ___ \text{ GB}$$

$$\text{ZeRO-2: } 2\Psi + 14\Psi/N = ___ \text{ GB}$$

$$\text{ZeRO-3: } 16\Psi/N = ___ \text{ GB}$$

GIVEN

$\Psi = 13 \times 10^9$, N $= 8$ GPUs, at most 60 GB of model state per GPU. GB $= 10^9$ bytes.

QUESTION

Which choices fit in 60 GB?

Exercise B: traffic per step

DP, ZeRO-1, ZeRO-2: $2 \cdot (N-1)/N \cdot S = ___$ GB

ZeRO-3: $3 \cdot (N-1)/N \cdot S = ___$ GB

GIVEN

S $= 2\Psi$ bytes (bf16) $=$ 26 GB. NVLink: 450 GB/s per direction.

QUESTION

How long does each take over NVLink? Combine with Exercise A: which one do you pick, and why?

Source: H100 datasheet (NVLink 900 GB/s bidirectional = 450 GB/s per direction).

Solutions A: memory per GPU

DP: $16\Psi = 208$ GB

ZeRO-1: $4\Psi + 12\Psi/\mathcal{N} = 71.5$ GB

ZeRO-2: $2\Psi + 14\Psi/\mathcal{N} = 48.75$ GB

ZeRO-3: $16\Psi/\mathcal{N} = 26$ GB

Check

Under 60 GB: **ZeRO-2** and **ZeRO-3** only.

Solutions B: traffic per step

DP, ZeRO-1, ZeRO-2: $45.5\ \text{GB} \rightarrow \approx 0.10$ seconds

ZeRO-3: $68.25\ \text{GB} \rightarrow \approx 0.15$ seconds

Check

Pick **ZeRO-2**: it fits at DP's traffic. ZeRO-3 only if you need the extra memory.

Source: H100 datasheet (NVLink 900 GB/s bidirectional = 450 GB/s per direction).

How to measure it

```
torch.cuda.reset_peak_memory_stats()
loss = F.cross_entropy(model(x), y)
loss.backward()
opt.step()
peak = torch.cuda.max_memory_allocated()
print(rank, peak / 1e9, "GB")
```

- Peak is **above** the formula: activations, buffers.
- Profiler: count the collectives.

In our pipeline

Compare DP and FSDP on the **same** batch: the **difference** should match the formulas.

Source: PyTorch documentation, `torch.cuda.max_memory_allocated`; `torch.profiler`.

Key takeaways

- Plain DP stores **N copies** of 16Ψ; only $1/N$ of it is distinct.
- **ZeRO-1/2** shard the optimizer state and gradients: **same traffic** as DP.
- **ZeRO-3 / FSDP** also shard the parameters: $\mathbf{16\Psi/N}$ for $\mathbf{1.5\times}$ the traffic.
- FSDP gathers each unit **just in time** and **prefetches** to hide the gathers.
- Pick the **cheapest** stage that fits; shard only what does not fit.

Next time, and readings

When	What
This week	T2 checkpoint; Lab 4 due
Mar 13–21	Spring break
Week 10, Lecture 1	Tensor parallelism: splitting one layer across GPUs

READING

Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020 — Sections 5 and 7. PyTorch documentation: fully_shard.